LIGHTWEIGHT LIGHTING

A browser engine with a tactile Zelda-like look

Research map, rendering architecture, implementation priorities and a linked primary-source bibliography.

Recommendation: build a compact raster renderer with smooth simplified PBR, a stable shadowed sun, baked or relightable indirect lighting, sparse probes for moving objects, and a small reflection-probe system. Add dynamic GI and stochastic many-light rendering only when a representative scene demonstrates the need.

The specific reference is Nintendo's new Ocarina of Time remake for Switch 2, released November 5, 2026. Nintendo describes a tactile fantasy presentation drawing on puppetry and practical-effects films. That favors material response, coherent soft fill, clean contact and shadow stability. The sources reviewed do not establish Nintendo's exact lighting implementation. [D16] [D17]

Build first	Add when justified	Keep as research
Stable cascaded sun shadows; deterministic PCF	Directional lightmaps; better probe visibility	DDGI, surfel GI, voxel cone tracing
RGB lightmaps + SH probes; coherent reflection probes	GTAO; local LTC lights; clustered light lists	Radiance Cascades and Split Radiance Cascades
Linear lighting; controlled exposure; spatial AA; fog	WishGI or HHD; authoring light optimization	Stochastic many-light shading; server-assisted GI

This report surveys 76 selected papers, conference/course talks, original engineering articles, specifications and implementations. It covers the useful technique families rather than claiming an exhaustive bibliography of every lighting publication. Priorities and budgets are engineering recommendations; historical paper timings are not browser benchmarks.

Contents

1. What the target actually requires 3

2. A lighting model that can evolve 3

3. Choose the browser rendering architecture 4

4. Direct light and shadows: first visual priority 6

5. Indirect diffuse light: move work out of the frame 7

6. Recent research with unusually good relevance 8

7. Reflections, AO, atmosphere and image stability 9

8. Dynamic GI: compare complete systems 10

9. Browser cost model and proposed quality tiers 12

10. From-scratch engine and asset architecture 14

11. Editable worlds, day/night and validity 15

12. Implementation order and validation 16

13. Annotated primary-source catalogue 17

How to use the report. Read Sections 1-4 to choose the visual and runtime baseline, Sections 5-8 to select lighting features, and Sections 9-12 to design the engine and validate performance. The final catalogue links every source and identifies its role and limitations. Reference IDs such as [I01] are clickable.

Scope assumptions. WebGPU is the primary target. Mobile browsers and integrated GPUs deserve a lower tier; desktop browsers can expose enhancements. No fixed minimum device or required frame rate was supplied. The report includes both published static worlds and editable/generated worlds; the latter need explicit lighting invalidation.

1. What the target actually requires

The implementation goal is a convincing stylized 3D world with readable forms and rich materials. Treat the rendering style as a coordinated choice across geometry, normals, textures, lighting, atmosphere and camera response. Hard toon bands and thick outlines should be optional material effects, because they are only one subset of stylization.

Nintendo's art interview emphasizes selective detail and deliberately designed spaces. My inference for a browser engine is to concentrate lighting resolution on playable surfaces, faces, doorways and important silhouettes; inaccessible backgrounds can use simpler illumination and atmospheric depth. This is a proposed application of the art direction, not a claim about the remake's renderer. [D16]

Visual property	Low-cost implementation direction	Common failure
Readable, tactile surfaces	Broad diffuse shading, roughness variation, restrained normal detail	Tiny noisy highlights and noisy shadow filters
Soft colored bounce	Baked irradiance, directional lightmaps or local SH	Uniform ambient brightness that illuminates caves
Grounded characters	Matching probe lighting, sun shadows, small contact detail	Characters with a different exposure or ambient palette
Dramatic rooms	One or two strong key lights, local reflection probes, selective LTC	Many overlapping lights with every light casting shadows
Landscape scale	Directional sun, distance/height fog, economical sky	Expensive volumetrics before basic value/color composition works
Clean motion	Stable shadows, filtered textures, deliberate AA policy	Shimmer, history ghosts and repeated exposure changes

The TF2 paper is useful for diffuse remapping, masked rim response and readable silhouettes. Gooch shading supplies a simple warm/cool orientation control; Guilty Gear demonstrates how far stylization can move into authored normals and asset choices. Use these as a palette of controls, rather than a requirement to copy their different art styles. [D12] [D13] [D14]

2. A lighting model that can evolve

Separate the production of lighting data from its use in a material. A surface should be able to query indirect diffuse and specular light regardless of whether an offline bake, an authored probe, a background update or a later GI system produced the data.

```
diffuse_E = query_irradiance(position, geometric_normal,
                             shading_normal, lighting_region)
prefiltered_L = query_prefiltered_radiance(
    position, reflected_view, roughness, lighting_region)
indirect_specular = evaluate_environment_BRDF(
    prefiltered_L, NdotV, roughness, F0)

color = emissive
      + shadowed_direct_diffuse_and_specular
      + diffuse_albedo / PI * diffuse_E * residual_diffuse_AO
      + indirect_specular * specular_visibility
```

This is an architectural decomposition, not a complete BRDF implementation. In particular, residual AO means occlusion missing from the chosen indirect representation. Applying another full-strength occlusion term to already occluded baked lighting can double-darken the scene. Emissive appearance also does not automatically illuminate neighboring objects: supply an analytic light, a bake contribution or a GI update explicitly. [R19]

Define conventions before optimizing

- Specify whether each cache stores incident radiance, diffuse irradiance E, E/PI, or already reflected light. Record whether the receiver albedo has been applied.
- Choose one world coordinate convention, SH coefficient order, normal basis and physical length scale. Distinguish geometric normals used for visibility from detailed shading normals.
- Document the roughness parameterization shared by direct specular shading, the environment prefilter and the BRDF lookup table.
- Perform lighting in linear color. Decode color textures correctly; keep normal/roughness/data textures in their intended numeric space; tone-map and encode output once.
- Decide whether a bright sun disk in the environment is included in the explicit sun light, the environment contribution, or a split of both. Avoid counting the same contribution twice.

Karis's UE4 notes and Filament provide practical starting points for a small diffuse-plus-GGX material model and consistent environment lighting. A physically motivated base is compatible with an artist-controlled diffuse curve, shadow palette, foliage transmission approximation and optional rim mask. These controls should have defined defaults so every material does not become a separate renderer. [I21] [R19]

3. Choose the browser rendering architecture

Start with forward shading and short light lists

Use CPU visibility culling and small per-object or spatially selected light lists first. A scene containing a sun and a few lanterns does not automatically benefit from a compute pass and thousands of cluster records. Instrument the number of lights actually shaded per fragment; the count of lights elsewhere in the world is a poor proxy for cost.

Forward+ builds tile light lists before forward material shading. Clustered shading adds depth grouping, reducing false overlap when a tile contains nearby foliage and distant buildings. These techniques are useful upgrades when local-light overlap becomes significant. The original papers establish the approach; the break-even point is a property of your renderer and content. [B01] [B02]

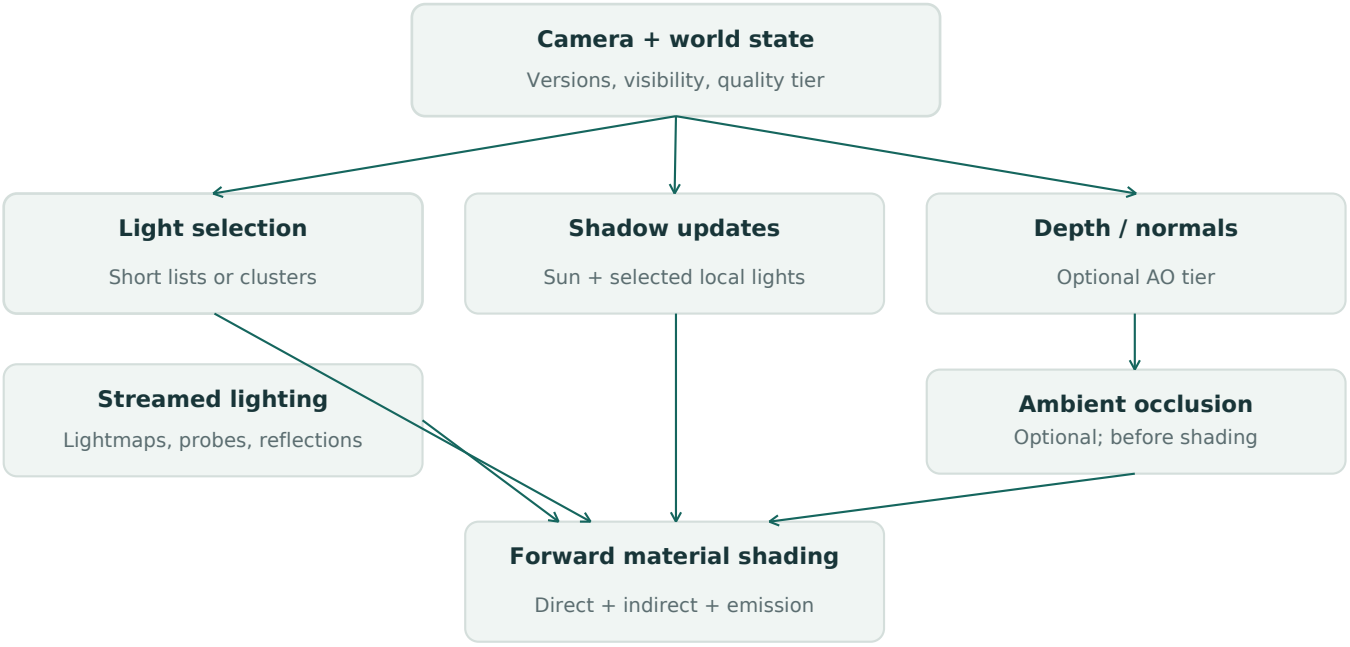
If transparent materials reuse these lists, do not restrict their light coverage using only opaque depth bounds. Build conservative lists covering transparent depths, or provide a separate transparent-light selection path.

Architecture	Useful situation	Cost to account for
Simple forward	Few influential lights; broad compatibility; material flexibility	CPU selection and repeated light work in overdraw
Clustered forward	More overlapping lights; depth discontinuities; transparent materials	Grid construction, light-index memory, dispatches and overflow handling
Deferred shading	Heavy light overlap or an effect pipeline already requiring a G-buffer	Attachments, bandwidth, transparency path and MSAA complexity

Architecture	Useful situation	Cost to account for
Stochastic many-light	Unpredictable creator scenes with very high overlap	Sampling structures, shadow updates, noise, history and denoising

Do not assume all deferred rendering is slow on mobile. Tile-based hardware can retain intermediate data cheaply under the right API and pass structure. Conversely, native subpasses, framebuffer fetch and tile shaders must not be assumed available through your portable browser backend. Keep the measured render graph, rather than the label "forward" or "deferred," at the center of the decision. [B04] [B05]

A concrete frame graph



Then sky / water / transparency -> fog / bloom / tone mapping / AA

For the lean tier, omit screen-space AO and the depth/normal prepass unless another measured need justifies it. For the AO tier, generate depth and optional normals, evaluate AO, then shade the opaque forward pass using that AO. An alternative is to preserve a separate indirect-light contribution for a later composite. Multiplying completed scene color by AO incorrectly darkens direct light and emission. [R19]

A depth prepass must use the same deformation, alpha test, camera jitter and geometry selection as the opaque pass. Otherwise the optimization becomes a source of holes, bad edges and lighting discontinuities. When combining MSAA with screen effects, define which depth samples the effects consume; do not assume every depth format has an automatic resolve matching color.

WebGPU features and fallback policy

Current WebGPU supports a portable rendering/compute model, while optional features and Compatibility Mode change available limits. Negotiate the actual adapter. Keep f32 fallbacks for shader-f16, avoid mandatory subgroup operations, and use timestamp-query only when supported. The reviewed standard has no portable hardware-ray-tracing baseline. [B06] [B07]

The reviewed specification permits sample counts 1 or 4; 4x MSAA on rgba16float requires core-features-and-limits. A single-sample HDR plus spatial-AA path is a useful compatibility choice. Compatibility adapters can also lack vertex-stage storage buffers, which matters when implementing vertex-based lighting reconstruction. [B06]

Consider WebGPU Compatibility Mode before committing to a full second graphics backend. If WebGL2 audience reach warrants the maintenance cost, provide CPU-selected lights, conservative formats, baked/probe lighting and spatial AA there. Advanced effects need not have identical implementations. Native wgpu capability is not proof that the same path will work in a browser. [B07] [B09]

4. Direct light and shadows: first visual priority

Stable cascaded shadows with deterministic PCF

Begin with one directional sun and two cascades as a proposed starting setting. Cascade count, shadow distance, map size and filter width should be separate quality controls. PSSM distributes resolution along camera depth. Stable projection extents and texel-snapped translation help prevent crawling under small camera movements. [D02] [D03]

- Cull shadow casters against the light/receiver region, including off-camera geometry that can cast into the visible area.
- Tie bias to the light projection and texel scale; expose diagnostic views for depth bias, normal offset and cascade selection.
- Use a transition band between cascades and verify consistent bias and softness on both sides.
- Use shadow-specific mesh LODs. Budget grass and leaves separately because they can be redrawn for several light views.
- Cache only when the light, caster, deformation and projection dependencies remain valid. A rotating sun or wind animation can invalidate an apparently static cache.

PCF averages depth-comparison results, not raw depths followed by one comparison. The Witness's shadow article is particularly valuable: randomized filtering was conspicuous against its smooth stylized surfaces, and the team pursued compact weighted filtering. That is a strong reason to evaluate deterministic PCF before introducing temporal noise. [D01] [D04]

Shadow method	What it changes	Recommendation
PCF	Filters binary depth comparisons	Default; prioritize stable projections and a compact kernel
PCSS	Adds blocker search and variable filter width	Optional key-light upgrade; extra samples can be expensive [D05]
VSM / EVSM	Filters depth moments or warped moments	Compare when wide filtering is important; handle bleeding and precision [D06] [D07]
Moment shadow maps	Uses four moments and an optimized visibility bound	Later alternative with more involved reconstruction [D08]
Screen-space shadows	Tests visibility against camera depth	Fine-detail supplement; retains world-space shadow fallback [D09]

Variance shadow maps and UE5-style virtual shadow maps are different techniques despite sharing an acronym. A virtual page system introduces allocation, residency, invalidation and filtering work; it should solve a demonstrated shadow-resolution problem before entering this small-engine baseline.

Local lights and area lights

Separate the number of visible lights from the number allowed to update shadows. A conventional point-light shadow cubemap has six faces. A room full of unshadowed selected lights and a room full of shadow-casting point lights are very different workloads. Use a dedicated update budget and stable priority rules so lights do not visibly flicker between shadowed and unshadowed states.

LTC approximates a material lobe using a transformed cosine distribution, enabling an analytic polygonal-light integral. It is attractive for broad window highlights and theatrical interior fill. It solves unoccluded light-shape shading; area-light visibility and soft shadows remain separate. Start with rectangles before line/disk extensions. [D10] [D11]

Low-cost artistic tools also matter: projected light cookies, controlled attenuation, a few selected fill lights, blob/contact approximations for the lowest tier, and an emissive-to-proxy-light authoring rule. Label approximations as such so authors understand what will react when geometry moves.

5. Indirect diffuse light: move work out of the frame

Lightmaps for stable architecture

For a fixed published scene, a low-resolution RGB irradiance lightmap is an excellent baseline. A baker can spend substantial work on multibounce light transport once, while runtime work is predominantly a texture lookup. The engineering cost moves into UV charts, atlas packing, padding, mip generation, compression, streaming and invalidation. Frostbite's precomputed-GI presentation is an unusually useful guide to these details. [I03]

Use a second UV set, chart-aware dilation and mip generation, consistent texel density, and lighting pages that stream independently of material textures. Verify geometry-LOD transitions against the bake. If the sun is dynamic, exclude its direct contribution from the lightmap; any baked sun bounce still belongs to its baked lighting scenario.

A single RGB value does not describe how irradiance changes with the material normal. RNM stores a small directional basis; SH and hemispherical bases offer alternatives. These enable detailed normal maps over comparatively coarse lighting. They cost more data and reconstruction than a scalar or RGB ambient term. [I02] [R15]

Spherical harmonics for moving objects

SH is a compact angular representation, not a GI solver. Degree 1 uses four coefficients per color; degree 2 uses nine. Degree 2 therefore uses nine RGB coefficients, or 27 scalar values in total. The original irradiance paper explains why diffuse convolution is well approximated with low angular frequencies. This does not mean illumination is spatially smooth across a doorway or wall. [I01]

```
E(position, normal) ~= sum_i c_i(position) * Y_i(normal)
L_diffuse = diffuse_albedo * E / PI

# Only use this convention if c_i already stores irradiance.
# Raw incident-radiance SH needs cosine convolution first.
```

Begin with a global SH environment and then add sparse local probes. For a small moving character, CPU interpolation can supply one or a few sets of coefficients; a large object spanning a doorway needs more spatial samples. Tetrahedra support irregular probe placement with four interpolation vertices, while regular grids provide a simpler addressing scheme. Neither makes interpolation automatically respect walls. [I07] [R14]

- Place more samples near openings, stairs and strong light transitions, with fewer in broad uniform outdoor regions.
- Add region identifiers, validity masks and conservative fallback before attempting a sophisticated learned visibility model.
- Provide a visualization of probe positions, weights and contributing regions. A bright leak must be traceable to the samples that produced it.
- Keep the baked and moving-object color/exposure conventions identical. This is central to making a character belong in the environment.

Dynamic time of day without full dynamic GI

The Ghost of Tsushima approach is a strong reference: store sky visibility/transfer information and relight it from the current sky, with deliberately approximate bounced components. The sharp direct sun remains a separate shadowing problem. A changing sky does not update visibility after a wall is removed. [I09]

For SH, the product of sky radiance and visibility is an angular-function product. It generally requires a proper SH product/projection or a chosen approximation; multiplying matching coefficients element by element is not the correct operation. Use a simple reference integrator to validate the result. [R14]

A smaller implementation can blend two or several baked lighting scenarios in linear lighting space. This is useful for day/night or authored transitions but does not reproduce every moving shadow or arbitrary edit. UberBake shows how anticipated interactions such as switched lights and doors can be incorporated into precomputed lighting. The state space and storage must stay bounded. [I19]

6. Recent research with unusually good relevance

WishGI, SIGGRAPH 2025

WishGI associates vertices with a compact fitted SH probe representation, shifting reconstruction toward vertex work and avoiding lightmap UVs and an extra render pass. It is a promising experiment for stable assets under memory pressure. Mesh-local associations can be reused, but illumination values still depend on each instance's surroundings. [I06]

Its important caveat here is vertex density: large sparse walls or terrain can require subdivision or a different lighting representation. The paper's native-device performance varies with device and LOD. Evaluate total mesh bytes, interpolation work, shading cost and quality together; do not treat its strong lighting-memory results as a universal browser speedup. [I06]

HHD and Neural Light Grid, SIGGRAPH 2024

HHD (Hemisphere and Highlight Direction) targets compact directional lightmaps and better blending behavior than older AHD representations. Use ordinary RGB or SH lighting as a reference before adopting the compression. The earlier Directional Lightmap Encoding Insights explains why blending nonlinear direction/intensity parameters can change brightness. [I04] [I05]

Neural Light Grid targets where and how probes should contribute, reducing visibility/interpolation artifacts while keeping runtime reconstruction practical. Its useful lesson is to spend learning and optimization in precomputation and reconstruct from a compact representation. It should follow a working conventional probe system; it is not a requirement to run a general neural GI model per pixel. [I10]

LightOpt, SIGGRAPH 2026

LightOpt optimizes the light setup itself, removing, merging and inserting lights while preserving a target appearance; it also considers time-of-day setups. This can reduce runtime work without replacing your renderer. For creator worlds, keep authored lights as the editable source and generate a separate optimized runtime representation. Preserve gameplay-significant lights, animation intent and interaction semantics. [R10]

Stochastic Tile-Based Lighting in HypeHype, 2025

This production talk is especially relevant to unconstrained creator scenes. It samples light candidates through tile reservoirs and shades a small selected set, using shadow maps rather than requiring hardware ray tracing. Its implementation includes a G-buffer, depth hierarchy and separate shadow work; temporal artifacts and denoising remain important tradeoffs. The results are not browser benchmarks. [B03]

My recommendation is a separate many-light experiment after a deterministic renderer exists. Compare the whole frame, including shadow updates and temporal reconstruction. A bounded final sample count does not make the entire lighting pipeline constant-cost.

Research	What it changes	When to try it
WishGI [I06]	Static-lighting representation and reconstruction	Lighting download/residency dominates and assets have suitable density
HHD [I05]	Directional lightmap encoding	Normal response or scenario blending exposes encoding limits
Neural Light Grid [I10]	Probe contribution/visibility representation	Conventional probe leaks remain a major art problem
LightOpt [R10]	Offline light authoring/optimization	Many redundant local lights cost significant shading time
HypeHype [B03]	Runtime many-light estimation	Unpredictable heavy overlap defeats deterministic selection

7. Reflections, AO, atmosphere and image stability

Reflection probes before screen-space reflections

Use one environment cubemap and a small number of local probes. Prefilter their radiance for material roughness and combine that with a BRDF integration lookup. Ordinary mipmap generation is not the required specular convolution. Local influence volumes and box-style parallax correction can make room reflections much more coherent. [I21] [I22]

Capture updates are separate from cheap sampling. Spread updates over time, prioritize probes near the camera and meaningful lighting changes, and avoid seams from inconsistent exposure. Water can begin with environment reflection and stylized highlights; a selectively updated planar reflection is a later option for important calm surfaces. Planar reflection renders another view, so it has a geometry/visibility cost.

SSR is useful when visible scene detail noticeably improves water or glossy floors, but it cannot see hidden or off-screen objects. McGuire and Mara's screen-space ray-tracing paper is the core implementation reading. Blend to probe reflections on a miss rather than fading to an obviously dark failure. [R04]

AO as a controlled contact/detail layer

SAO and GTAO are the strongest first readings. SAO emphasizes data access and predictable work; GTAO improves the occlusion model and related indirect-light approximations. XeGTAO provides a practical integration reference. Prefer an optional low-resolution effect with edge-aware reconstruction, small intended radius and an explicit full-pass cost. [R01] [R02] [R03]

Do not use screen-space AO to repair a fundamentally incorrect indoor/outdoor lighting field. Dark-room leaks require visibility, probe-region or bake fixes. Avoid combining multiple occlusion terms that encode the same cavity. Add motion history only if its quality benefit exceeds the extra motion-vector and rejection work.

Atmosphere and fog often have high visual payoff

Implement height/distance fog and a designed sky first. A simple moving cloud-shadow texture can add broad variation to outdoor light. Keep the sky color used for ambient illumination consistent with the visible sky, even if both are artist-directed approximations.

Hillaire's 2020 atmosphere work is the next upgrade: compact lookup tables support sky and aerial perspective without a large per-pixel atmospheric integral. Full volumetric lighting introduces another grid, light injection and integration system; the Frostbite 2015 talk is useful once shafts and local fog become visually necessary. [R05] [R06]

Anti-aliasing is part of the lighting design

Smooth stylized surfaces make instability conspicuous. Establish spatial AA or supported 4x MSAA first; SMAA is a useful reference. If noisy shadows, AO or GI require temporal accumulation, then velocity generation, depth/normal rejection, history clamping, camera cuts and resolution changes become core renderer responsibilities. [B13] [R07] [B12]

Animate wind and skinning consistently across depth, shadow and velocity passes. Material edits and disappearing objects need history invalidation too. A still image can hide a bad temporal solution; review slow pans, rotations, disocclusions and low-frame-rate motion.

Specular aliasing also needs material filtering. Toksvig and LEAN show how normal variation informs stable highlights. For this target, deliberate roughness and reduced subpixel normal detail may be a cheaper first control than an elaborate temporal filter. [R08] [R09]

8. Dynamic GI: compare complete systems

Dynamic indirect lighting can be valuable for moving lamps, destruction and generated worlds. The decisive question is which changes must affect bounce immediately. Many scenes can respond instantly through direct lights and shadows while indirect illumination converges more slowly.

Family	Representation / work	Best reason to investigate	Main risk
SSGI / SSIL [R22]	Visible depth, normals and lit color estimate local bounce	Small near-field enhancement using screen-space data	Missing surfaces, temporal artifacts and double counting

Family	Representation / work	Best reason to investigate	Main risk
Reflective shadow maps [I15]	Reflective flux/normal/position in light view; sample secondary lights	Small one-bounce sun or key-light experiment	Incomplete secondary visibility and sampling cost
LPV [I13]	Inject and propagate low-frequency lighting through volumes	Very soft approximate bounce in forgiving geometry	Thin-wall leaks and repeated volume traffic
Voxel cone tracing [I14]	Voxelized radiance hierarchy plus cone queries	Existing voxel infrastructure can be reused	Voxelization, memory, filtering and updates
DDGI [I11] [I12]	Ray-updated irradiance and directional distance information	Dynamic probe field with explicit update budget	Traversal, hit shading, temporal lag and probe leakage
Sparse local reconstruction [R17]	Precompute local transport/visibility; update lighting samples	Mostly static geometry with changing lights	Precomputation assumptions and complete update cost
Surfel GI / Lumen [R13] [R12]	Scene representations plus caches, tracing and reconstruction	Long-term fully dynamic rendering architecture	Large dependency chain and substantial engineering
Radiance Cascades [I16]-[I18]	Distance intervals trade spatial for angular detail	Research into multiscale diffuse transport	3D storage, tracing and update behavior need evidence

Screen-space indirect lighting as a limited enhancement

SSGI/SSIL can add a local diffuse bounce from visible scene data and may share preparation with AO. The visibility-bitmask paper improves how a horizon-style method handles thin surfaces. It cannot reconstruct unknown geometry. Keep a baked/probe fallback and define whether the effect replaces part of that estimate or adds a bounded residual, so existing bounce is not counted twice. Include reconstruction and any temporal filtering in the measured cost. [R22]

DDGI query cost is only part of the problem

DDGI's separation between updating a field and shading from it is useful even when the first producer is an offline baker. But dynamic production still needs rays, a scene representation, hit shading and temporal handling. No portable browser RT baseline is established by the reviewed WebGPU sources; software traversal must earn its cost. [I11] [I12] [B07]

An illustrative update budget shows the tradeoff: 4,096 probes times 128 rays is 524,288 rays for a full sweep. Updating one-sixteenth per frame is 32,768 rays per frame, with a 16-frame revisit interval: about 267 ms at 60 Hz, before additional convergence. These are arithmetic examples, not a proposed universal configuration or a speed estimate.

Radiance Cascades deserves careful scope

The original author manuscript is a living WIP, and it explicitly discusses the large storage burden of the base 3D volume. Holographic Radiance Cascades is a 2D paper. Split Radiance Cascades, revised October 2026, addresses 3D with sparse world-space probes and ray splitting; its abstract is promising but does not establish a browser/mobile budget. Keep these on a research branch. [I16] [I17] [I18]

Other advanced directions

ReSTIR is important for sampling many lights and reusing estimates over space and time. It does not supply visibility evaluation, a browser ray-tracing API, or a free denoiser. Its ideas can inform shadow-map-based many-light systems, but an intentional scene with a handful of lights has little reason to inherit the full complexity. [R11]

Recent mobile-GI talks are worth tracking, including the 2026 Moving Mobile Graphics contribution. Native mobile hardware demonstrations and browser portability remain separate questions. Similarly, neural radiance caches can shift transport work rather than remove it; do not assume that a small neural representation has a small total training, tracing and bandwidth cost. [R20]

Server-assisted probe streaming is a strategic option for shared worlds: keep camera visibility and direct light local while a server updates indirect light. Stengel et al. study that split. It introduces server capacity, network and update-latency decisions, and its codec pipeline needs separate browser qualification. It is an optional producer for the same lighting interface, not a reason to make local rendering wait on the network. [R18]

9. Browser cost model and proposed quality tiers

Count pixels, bytes, overlap and updates

A useful first cost model is: visible shaded samples multiplied by material/light work, plus shadow rendering, screen-space passes and lighting-data updates. Overdraw multiplies more than one term. Device-pixel ratio can silently increase internal resolution; choose internal rendering size independently from CSS size and keep the interface crisp at display resolution. [B09]

At 75% scale in each dimension, there are 56.25% as many pixels. That reduces resolution-dependent work; it does not reduce shadow maps, CPU simulation or geometry processing by the same fraction. A dynamic-resolution controller should use hysteresis and avoid resetting every temporal history on small changes.

Resource / example	Raw storage or traffic	Interpretation
1280 x 720 RGBA16F scene color	7.03 MiB	One single-sample attachment
1280 x 720 32-bit depth	3.52 MiB	Excludes implementation overhead
Four 32-bit G-buffers at 1080p	31.64 MiB	Excludes depth and other targets
Those four buffers, one write + read at 60 Hz	3.98 GB/s	Logical uncompressed traffic; real traffic varies

Resource / example	Raw storage or traffic	Interpretation
Three 2048-square depth32float shadow maps	48 MiB	Caster rendering and sampling are extra costs
One 2048-square RGBA16F lightmap + mips	About 42.67 MiB	Before compression and other allocations
32-cubed probes, 27 packed half values each	1.6875 MiB	Before padding, visibility and lookup structures
128-square RGBA16F cubemap, six faces + mips	About 1 MiB	Only one reflection capture representation

These are exact format arithmetic or full-mip approximations, not measured resident-memory guarantees. Driver padding, render-target compression, cache behavior, GPU allocation rules, MSAA, transient buffers and upload copies change actual use. Do not assign a fixed byte size to the implementation-dependent depth24plus format.

Avoid an unexpectedly large cluster buffer

At 1280 x 720 with 16-pixel XY tiles and 24 depth slices there are 80 x 45 x 24 = 86,400 clusters. Reserving 64 four-byte light indices for every cluster consumes about 21.09 MiB before metadata. Compact offsets/counts with packed indices, coarse grids or tighter validated limits can matter more than a small BRDF optimization. Keep overflow visible in diagnostics and use a stable ranked fallback rather than silently dropping arbitrary lights.

Initial envelopes to test

Tier	Suggested starting configuration	Engineering target
Broad mobile	Around 720p dynamic; 1-2 modest cascades; simple probes/lightmaps; spatial AA; no required AO	30 FPS; aim below about 25 ms sustained GPU time
Strong mobile / integrated laptop	720p-1080p dynamic; 2-3 cascades; optional half-resolution AO; richer local reflections	60 FPS if sustained; aim around 10-13 ms GPU time
Desktop enhancement	Start at 1080p; add selected higher-cost effects after profiling	60 FPS; preserve frame-pacing headroom

These are proposed experiment settings, not measured predictions. CPU and GPU execution overlap, so do not naively add their separately reported times into a frame budget. Evaluate the bottleneck and end-to-end pacing. Thermal behavior, scene content, browser and driver differences can change the winning configuration.

10. From-scratch engine and asset architecture

Keep four responsibilities separate

Module	Owns	Stable contract
Scene lighting	Lights, environment, regions, gameplay state	Which sources and interactions currently exist
Lighting producer	Baker, scenario blender or future dynamic GI	Generate versioned irradiance/reflection data
Lighting residency	Page tables, probe chunks, mips, validity	Give shaders a safe current representation or fallback
Material shading	BRDF, stylistic controls, direct/indirect composition	Consume consistent lighting quantities

For a Wasm CPU core, keep GPU shaders as an explicit WGSL/backend layer. Bulk-upload contiguous scene data and avoid many tiny JS/Wasm crossings. Native and browser builds can share scene formats, math and validation while retaining different capability profiles. Use a single source of truth for binding layouts and padding; a compact CPU struct is not automatically a valid GPU storage layout. [B08] [B15]

Compile a small finite set of material variants before they become visible, use asynchronous pipeline creation, and cache reusable resources. Instancing and stable render bundles can reduce repeated submission work, but measure them against the update rate of your scene. Avoid synchronous readback for either features or profiling; collect timings later. [B06] [B14]

A practical lighting chunk

```
LightingChunk {
  world_bounds, region_id, content_version
  geometry_dependencies, material_dependencies, light_dependencies
  lightmap_page_handles, available_mips
  probe_positions, irradiance_coefficients, validity
  optional_sky_visibility, scenario_identifiers
  reflection_volumes, reflection_capture_handles
  bake_state, fallback_policy
}
```

Lighting chunks need overlap or border samples so interpolation remains continuous while neighbors stream. A probe page becoming unavailable must fall back deliberately; sampling uninitialized data or a default white probe is not a valid residency policy. Keep allocation and validity stable across camera motion to avoid flicker.

Treat download compression and GPU compression separately

KTX2 and Basis Universal-related workflows can reduce transmission costs and transcode into device-supported texture formats. GPU residency is determined by the resulting representation, not the network file size. Lighting data also needs correct linear decoding and sufficient range; an LDR color encoding is not interchangeable with HDR irradiance or signed SH coefficients. [B10] [B11]

Stream coarse useful lighting first. Budget temporary decompression and upload buffers as well as final GPU storage. Use conservative default materials while detailed assets arrive, and keep their exposure/color behavior close enough that streaming does not create large lighting flashes.

The baker is a tool, not a per-frame dependency

A from-scratch baker can begin with a simple path tracer, a BVH, next-event light sampling, diffuse transport and independently specified test scenes. It can run natively or on a build server even when the game renders entirely in the browser. Emit lightmaps and probes using the exact runtime conventions and validate both against a high-sample reference. Faster GPU preview can be added later. [I03]

Your first baker should produce predictable, inspectable data. Expose sample count, convergence/error estimates, chart padding, invalid texels and probe visibility. Incremental rebuilds and clear error reporting can save more iteration time than an advanced runtime GI system introduced too early.

11. Editable worlds, day/night and validity

For an editor or generated world, "mark the bake dirty" is incomplete: you must decide how the changed region looks immediately. Direct lights and shadows can respond this frame, while indirect lighting uses a safe fallback until updated data is ready. This is a proposed engine policy, not a claim that all cited GI methods support arbitrary editing.

- An edit changes the scene version and immediately updates direct rendering.
- Invalidate affected indirect dependencies, including neighboring receivers. Removing a wall can alter light beyond that wall's own chunk.
- Route changed geometry and affected samples to valid coarse probes, environment fill or a deliberate authored fallback.
- Rebuild local lighting asynchronously; preserve valid unaffected data where dependency tracking allows.
- Accept a result only if its geometry, material, light and bake-setting dependencies still match.
- Swap complete pages/fields atomically or blend compatible results. Never present half-written lighting records.

Different changes invalidate different data. Changing sky color can reuse sky visibility. Moving a wall invalidates visibility. Changing surface color can invalidate bounced lighting while leaving some geometric visibility valid. Moving the sun changes direct shadows and any sun-dependent bounce. Make these distinctions part of the lighting asset format.

Published worlds can afford high-quality background bakes and compact runtime delivery. Rapidly edited worlds may prefer probes and coarse relighting until publication. Device-adaptive representations, such as the direction discussed in Roblox's 2026 SLIM talk, are useful architecture reading for separating editable source content from efficient runtime content. [R21]

12. Implementation order and validation

Milestones with useful exit criteria

Stage	Build	Evidence to require
1. Correct image	Linear material inputs, simple diffuse/specular, environment fill, exposure, spatial AA	Known diffuse and roughness references; no double gamma or mismatched light units
2. Stable direct light	One sun, cascades, deterministic PCF, selected local lights	Slow camera pans and rotations without crawling; correct off-screen casters
3. Convincing indirect	RGB/directional lightmaps, SH probes, reflection volumes	Character crosses indoor/outdoor boundary without obvious leaks or exposure mismatch
4. Browser delivery	Streamed lighting pages, capability tiers, bulk uploads, pipeline preparation	No major frame spikes or memory growth while crossing world chunks
5. Selective refinement	AO, sky LUTs, better probe visibility, LTC, clustered lights as needed	Each feature passes a same-scene quality/cost comparison
6. Research branch	WishGI, HHD, LightOpt, NLG or dynamic-GI candidate	Documented bottleneck solved on representative target devices

Build a small but revealing scene suite

- An outdoor courtyard with a character, trees, a distant mountain and moving sun: shadow stability, foliage cost, atmospheric depth and LOD transitions.
- A bright exterior connected to a dark room through a narrow doorway: probe leakage, indirect gradients, exposure and reflection-region boundaries.
- A stylized dungeon with glossy props, fire, small lights and an emissive puzzle object: overlap, shadow budget, emission semantics and broad highlights.
- A water area with transparent effects and rapid camera motion: reflection fallback, overdraw, temporal disocclusion and post-processing.
- An editable wall/roof and material-color test: invalidation extent, stale-data fallback, rebuild acceptance and transition quality.

Measure CPU and GPU time distributions, not just mean FPS. Report p50/p95/p99 frame times, internal resolution, shadow dimensions, resident bytes, upload bytes, light overlap, draw counts, shader compilation events and device/browser versions. Run sustained mobile sessions long enough to see thermal behavior; record cold startup and streaming separately from steady-state rendering.

Debug views should include direct diffuse, direct specular, indirect diffuse, indirect specular, AO, cascade index, shadow bias, light-list occupancy, probe weights/validity, lightmap density, lighting version and temporal-history rejection. These are implementation tools with immediate value, rather than optional polish.

First reading order

Read the Nintendo art interview [D16], Filament [R19], The Witness shadow summary [D04], PSSM/stable cascades [D02] [D03], the SH irradiance paper and practical SH notes [I01] [R14], Frostbite precomputed GI [I03], Cupisz probes [I07], Tsushima lighting [I09], and UE4 IBL [I21]. Then choose newer work according to the bottleneck: WishGI/HHD for representation, NLG for leakage, LightOpt for redundant lights, or HypeHype for extreme overlap.

Recommended first shipped renderer: one stable shadowed sun; bounded local-light lists; a compact diffuse/specular material model; baked or relightable indirect light; moving-object probes; a few reflection probes; simple fog; deliberate color management and anti-aliasing. Preserve the lighting interfaces so later research improves the producer or representation without requiring a new material system.

13. Annotated primary-source catalogue

The entries distinguish papers from production talks, original technical articles, standards and implementations. "Implement first" and similar priorities are recommendations for this project. Living documentation was checked for this November 2026 report. Full texts were reviewed where accessible; some entries rely on primary abstracts or production summaries, as noted.

Indirect lighting, baking and reflections

[I01] An Efficient Representation for Irradiance Environment Maps
Ravi Ramamoorthi; Pat Hanrahan. 2001. SIGGRAPH 2001.
Use: Encode diffuse angular irradiance with four or nine coefficients per color. **Limit:** Does not solve spatial visibility or compute GI.
Open primary source | graphics.stanford.edu | Implement first

[I02] Shading in Valve's Source Engine
Jason Mitchell; Gary McTaggart; Chris Green. 2006. SIGGRAPH 2006 Advances in Real-Time Rendering course.
Use: Practical reference for radiosity normal mapping: bake directional illumination into a small tangent-space basis, then evaluate it using the material normal map. **Limit:** Directional storage increases bandwidth and data size over one RGB irradiance map.
Open primary source | cdn.fastly.steamstatic.com | Implement first / compare encodings

[I03] Precomputed Global Illumination in Frostbite
Yuriy O'Donnell. 2018. GDC 2018.
Use: Read the baker, atlas, filtering and SH-encoding workflow. **Limit:** Static data trades frame work for memory and authoring time.
Open primary source | media.gdcvault.com | Implement first

[I04] Directional Lightmap Encoding Insights
Peter-Pike Sloan; Ari Silvennoinen. 2018. SIGGRAPH Asia 2018 Technical Briefs.
Use: Explains why naively interpolating Ambient and Highlight Direction (AHD) parameters causes brightness artifacts, and proposes irradiance-preserving parameterizations and constrained fitting from SH. **Limit:** AHD models are nonlinear; interpolating parameters is not generally equivalent to interpolating the represented lighting.
Open primary source | arisilvennoinen.github.io | Optimize later

[I05] Hemispherical Lighting Insights from the Call of Duty Production Lessons
Thomas Roughton. 2024. SIGGRAPH 2024 Advances in Real-Time Rendering course.
Use: Introduces the Hemisphere and Highlight Direction (HHD) lightmap model, improved runtime blending over AHD, hemispherical occlusion, visibility-cone reasoning, and preservation of irradiance along the geometric hemisphere normal. **Limit:** Advanced encoding and baking constraints increase implementation complexity; establish a simple reference encoding first.
Open primary source | www.advances.realtimerendering.com | Optimize later; high-value modern reading

[I06] WishGI: Lightweight Static Global Illumination Baking via Spherical Harmonics Fitting
Junke Zhu; Zehan Wu; Qixing Zhang; Cheng Liao; Zhangjin Huang. 2025. ACM Transactions on Graphics 44(4), SIGGRAPH 2025; DOI 10.1145/3730935.
Use: Evaluate compact vertex/probe-associated static lighting without lightmap UVs. **Limit:** Vertex density and device/LOD tradeoffs matter.
Open primary source | arxiv.org | Prototype early for static assets

[I07] Light Probe Interpolation Using Tetrahedral Tessellations
Robert Cupisz. 2012. GDC 2012.
Use: Practical method for interpolating irregularly placed probes: locate a containing tetrahedron and blend its four vertex probes with barycentric weights. **Limit:** Tetrahedralization alone does not prevent interpolation through walls.
Open primary source | media.gdcvault.com | Implement first for moving objects

[I08] Large-Scale Global Illumination in Call of Duty
Ari Silvennoinen. 2021. SIGGRAPH 2021 Advances in Real-Time Rendering course.
Use: Production reading on compressed precomputed lighting, reconstruction directly from compressed data, visibility-based volumetric sampling, and constrained SH. **Limit:** Production-scale compression and visibility representations are substantial infrastructure.
Open primary source | research.activision.com | Optimize later for large worlds

[I09] Real-Time Samurai Cinema: Lighting, Atmosphere, and Tonemapping in Ghost of Tsushima
Jasmin Patry. 2021. SIGGRAPH 2021 Advances in Real-Time Rendering course.
Use: Study relightable sky visibility and deliberately approximate bounce. **Limit:** Geometry visibility still depends on the precomputation.
Open primary source | advances.realtimerendering.com | Implement early if time of day matters

[I10] Neural Light Grid: Modernizing Irradiance Volumes with Machine Learning
Michał Iwanicki. 2024. SIGGRAPH 2024 Advances in Real-Time Rendering course.
Use: Improve visibility-aware probe contribution with compact learned representations. **Limit:** Offline training/baking is substantial; avoid general per-pixel neural GI assumptions.
Open primary source | www.advances.realtimerendering.com | Advanced optimization

[I11] Dynamic Diffuse Global Illumination with Ray-Traced Irradiance Fields
Zander Majercik; Jean-Philippe Guertin; Derek Nowrouzezahrai; Morgan McGuire. 2019. Journal of Computer Graphics Techniques 8(2), 1-30.
Use: DDGI updates compact irradiance probes from rays and stores directional distance moments to support visibility-aware interpolation. **Limit:** Ray generation, scene acceleration, hit shading, temporal update, probe memory, and visibility lookup all have costs.
Open primary source | jcgt.org | Optional high tier / study visibility

[I12] Scaling Probe-Based Real-Time Dynamic Global Illumination for Production
Zander Majercik; Adam Marrs; Josef Spjut; Morgan McGuire. 2021. Journal of Computer Graphics Techniques 10(2), 1-29.
Use: Production extensions to DDGI: self-shadow bias, faster lighting transitions, probe state machines to skip ineffective work, and multiresolution cascaded volumes. **Limit:** These optimizations reduce an already complex GI system's cost; they do not eliminate tracing or temporal latency.
Open primary source | jcgt.org | Optional high tier

[I13] Cascaded Light Propagation Volumes for Real-Time Indirect Illumination
Anton Kaplanyan; Carsten Dachsbacher. 2010. ACM Symposium on Interactive 3D Graphics and Games (I3D 2010).
Use: Inject reflected light into SH-valued voxel lattices, propagate it through neighboring cells, and use cascades to cover larger regions. **Limit:** Coarse spatial discretization causes leaks through thin walls and blurs contact detail; propagation introduces approximation and additional passes.
Open primary source | cg.ivd.kit.edu | Experimental dynamic-GI candidate

[I14] Interactive Indirect Illumination Using Voxel Cone Tracing
Cyril Crassin; Fabrice Neyret; Miguel Sainz; Simon Green; Elmar Eisemann. 2011. Computer Graphics Forum / Pacific Graphics 2011.
Use: Voxelize geometry and radiance into a hierarchical representation, then trace widening cones through filtered levels to estimate diffuse, rough glossy lighting, and visibility. **Limit:** Dynamic voxelization, hierarchy maintenance, memory, filtering, and cone traversal form a large system.
Open primary source | research.nvidia.com | Research / optional high tier

[I15] Reflective Shadow Maps
Carsten Dachsbacher; Marc Stamminger. 2005. ACM Symposium on Interactive 3D Graphics and Games (I3D 2005).
Use: Augment a light-view shadow map with position, normal, and reflected flux, treating texels as virtual secondary light sources. **Limit:** Basic formulations omit or approximate visibility from secondary light to receiver, causing light leaks.
Open primary source | dl.acm.org | Small dynamic-GI experiment

[I16] Radiance Cascades: A Novel Approach to Calculating Global Illumination [WIP]
Alexander Sannikov. Undated living draft; accessed 2026-11-07. Author manuscript / work in progress.
Use: Study distance-dependent spatial/angular sampling. **Limit:** Living WIP; full 3D base storage remains a major consideration.
Open primary source | github.com | Research track

[I17] Holographic Radiance Cascades for 2D Global Illumination
Rouli Freeman; Alexander Sannikov; Adrian Margel. 2025. arXiv preprint 2505.02041.
Use: Constructs radiance transfer by combining short ray intervals within a multilevel probe system. **Limit:** Explicitly a 2D technique.
Open primary source | arxiv.org | Research; mainly 2D

[I18] Split Radiance Cascades: Real-Time Global Illumination via Sparse Radiance Probes
Rouli Freeman; Alexander Sannikov; Joe Pitt-Francis. 2026. arXiv preprint 2607.20384v2; revised 11 October 2026.
Use: Sparse world-space probes and ray splitting extend the approach to 3D. **Limit:** Primary abstract verified; full PDF unavailable in this review.
Open primary source | arxiv.org | Research track; prototype after baseline

[I19] The Design and Evolution of the UberBake Light Baking System
Dario Seyb; Peter-Pike Sloan; Ari Silvennoinen; Michał Iwanicki; Wojciech Jarosz. 2020. ACM Transactions on Graphics 39(4), SIGGRAPH 2020.
Use: Demonstrates that baked lighting can support bounded gameplay changes: individually switching lights and precomputing lighting changes for opening/closing doors. **Limit:** Works for anticipated, parameterized changes; arbitrary creator edits still invalidate precomputed transport.
Open primary source | cs.dartmouth.edu | Implement selected ideas for gameplay

[I20] Precomputed Radiance Transfer for Real-Time Rendering in Dynamic, Low-Frequency Lighting Environments
Peter-Pike Sloan; Jan Kautz; John Snyder. 2002. ACM SIGGRAPH 2002.
Use: Precompute how a static object's geometry transfers low-frequency incident light, then change the environment lighting at runtime using compact SH dot products or matrices. **Limit:** Precomputed transport assumes the baked geometric/material relationships.
Open primary source | www.microsoft.com | Study; selective use

[I21] Real Shading in Unreal Engine 4
Brian Karis. 2013. SIGGRAPH 2013 Physically Based Shading course.
Use: Practical split-sum image-based lighting: prefilter environment radiance across roughness levels and use a small BRDF integration lookup. **Limit:** Ordinary mipmaps are not a BRDF convolution.
Open primary source | cdn2.unrealengine.com | Implement first for specular IBL

[I22] Local Image-based Lighting With Parallax-corrected Cubemap
Sébastien Lagarde; Antoine Zanuttini. 2012. SIGGRAPH 2012 / Game Connection 2012 production presentation.
Use: Uses local influence volumes, cubemap blending, and proxy-geometry parallax correction to make inexpensive reflections fit rooms better. **Limit:** Box/sphere proxies are approximations; complex rooms and dynamic objects remain wrong in reflections.
Open primary source | seblagarde.wordpress.com | Implement early for interiors

Direct lighting, shadows and art direction

[D01] Rendering Antialiased Shadows with Depth Maps
William T. Reeves; David H. Salesin; Robert L. Cook. 1987. SIGGRAPH 1987.
Use: Percentage-closer filtering averages depth-test visibility rather than averaging depths before a single test. **Limit:** PCF antialiases and softens shadow boundaries but fixed-radius filtering does not reproduce general area-light penumbrae.
Open primary source | graphics.pixar.com | Core foundation

[D02] Parallel-Split Shadow Maps on Programmable GPUs
Fan Zhang; Hanqiu Sun; Oskari Nyman. 2007. GPU Gems 3, Chapter 10.
Use: Partition the camera frustum into depth ranges so a small number of sun-shadow maps allocates resolution more effectively. **Limit:** Extra cascades increase rendering and storage costs.
Open primary source | developer.nvidia.com | Implement early

[D03] Graphics Tech: Shadow Maps (part 1)
Jonathan Blow. 2010. The Witness production engineering article.
Use: Explains production use of Michal Valient's Stable Rendering of Cascaded Shadow Maps (ShaderX6, 2008): stable cascade extents, texel alignment, and atlas organization. **Limit:** Stability requires conservative projection bounds and sacrifices some usable shadow-map resolution.
Open primary source | the-witness.net | Implement early

[D04] Shadow Mapping Summary - Part 1
Ignacio Castaño. 2013. The Witness production engineering article.
Use: Study compact weighted PCF and practical bias handling. **Limit:** Randomized noise can stand out on smooth stylized surfaces.
Open primary source | the-witness.net | Highest-priority practical shadow reference

[D05] Percentage-Closer Soft Shadows
Randima Fernando. 2005. SIGGRAPH 2005 sketch.
Use: Blocker search estimates a penumbra size that controls a variable PCF filter, producing contact-hardening soft shadows. **Limit:** Additional blocker-search and filtering samples can be expensive.
Open primary source | download.nvidia.com | Optional higher-quality shadows

[D06] Variance Shadow Maps
William Donnelly; Andrew Lauritzen. 2006. ACM Symposium on Interactive 3D Graphics and Games (I3D).
Use: Stores two moments of the depth distribution so standard linear filtering can support a statistical visibility estimate. **Limit:** Light bleeding and numerical precision are important failure modes.
Open primary source | doi.org | Alternative shadow representation

[D07] Rendering Antialiased Shadows using Warped Variance Shadow Maps
Andrew Timothy Lauritzen. 2008. University of Waterloo master's thesis.
Use: Primary source describing exponential variance shadow maps (EVSM), which combine moments of two exponentially warped depth distributions to improve visibility bounds. **Limit:** Exponent choice, floating-point range, filtering, and format bandwidth need careful management.
Open primary source | uwspace.uwaterloo.ca | Advanced alternative

[D08] Moment Shadow Mapping
Christoph Peters; Reinhard Klein. 2015. ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games (I3D).
Use: Uses four depth moments for a tighter filtered visibility approximation; includes an optimized 64-bit-per-texel representation and implementation resources. **Limit:** More mathematical and numerical complexity than PCF, with additional channel bandwidth and prefiltering.
Open primary source | momentsingraphics.de | Advanced alternative

[D09] Screen Space Shadows (Bend Studio)
Graham Aldridge. 2023. Sony Creators Conference at SIGGRAPH 2023; studio presentation and example code.
Use: Production screen-space shadowing complements shadow maps with small or distant shadow detail; the studio provides slides and example code. **Limit:** Screen depth omits hidden and off-screen occluders.
Open primary source | www.bendstudio.com | Optional detail layer

[D10] Real-Time Polygonal-Light Shading with Linearly Transformed Cosines
Eric Heitz; Jonathan Dupuy; Stephen Hill; David Neubelt. 2016. ACM Transactions on Graphics / SIGGRAPH 2016.
Use: Fits BRDF lobes with transformed cosine distributions and integrates polygonal lights efficiently. **Limit:** The unoccluded area-light integral does not solve area-light shadows.
Open primary source | eheitzresearch.wordpress.com | Useful optional area lights

[D11] Real-Time Line- and Disk-Light Shading with Linearly Transformed Cosines
Eric Heitz; Stephen Hill. 2017. SIGGRAPH course: Physically Based Shading in Theory and Practice.
Use: Extends the LTC framework to line and disk sources, useful when light shape is an important artistic control. **Limit:** Lower implementation priority than rectangular lights; area visibility remains separate.
Open primary source | eheitzresearch.wordpress.com | Later area-light expansion

[D12] A Non-Photorealistic Lighting Model for Automatic Technical Illustration
Amy Gooch; Bruce Gooch; Peter Shirley; Elaine Cohen. 1998. SIGGRAPH 1998.
Use: Uses hue as well as luminance to communicate shape, reserving extreme values for highlights and contours. **Limit:** The technical-illustration aesthetic is not the new Ocarina remake's default look.
Open primary source | users.cs.northwestern.edu | Optional artistic control

[D13] Illustrative Rendering in Team Fortress 2
Jason Mitchell; Moby Francke; Dhabih Eng. 2007. International Symposium on Non-Photorealistic Animation and Rendering (NPAR).
Use: Connects art direction, silhouettes, diffuse-response warping, and controlled rim lighting to readable real-time scenes. **Limit:** Its specific illustrative style is different from tactile puppet/practical-effects imagery.
Open primary source | cdn.steamstatic.com | Core stylized-shading reference

[D14] GuiltyGearXrd's Art Style: The X Factor Between 2D and 3D
Junya Christopher Motomura. 2015. Game Developers Conference (GDC).
Use: A production case study showing that stylization depends on model, normal, shadow, and animation authoring as well as shaders. **Limit:** The deliberately 2D fighting-game illusion differs substantially from the user's Ocarina remake target and should not dictate the default renderer.
Open primary source | www.arcsystemworks.com | Optional stronger anime style

[D15] Change and Constant: Breaking Conventions with The Legend of Zelda: Breath of the Wild
Hidemaro Fujibayashi; Satoru Takizawa; Takuhiro Dohta. 2017. Game Developers Conference (GDC).
Use: Primary Nintendo production talk relevant to visual readability, art direction, and the interaction of game systems. **Limit:** It concerns Breath of the Wild, not the 2026 Ocarina remake.
Open primary source | www.gdcvault.com | Art and production context

[D16] Ask the Developer Vol. 24: The Legend of Zelda: Ocarina of Time - Part 2
Nintendo; interview with the Ocarina of Time development team. 2026. Official Nintendo developer interview, November 2, 2026.
Use: Grounds the requested tactile visual target and selective detail. **Limit:** Does not disclose a complete lighting implementation.
Open primary source | www.nintendo.com | Essential visual-target context

[D17] It's time. Experience The Legend of Zelda: Ocarina of Time, reborn exclusively for Nintendo Switch 2!
Nintendo. 2026. Official Nintendo launch announcement, November 5, 2026.
Use: Confirms November 5, 2026 launch exclusively for Switch 2. **Limit:** Release context, not a graphics architecture disclosure.
Open primary source | www.nintendo.com | Release verification context

Browser architecture and image stability

[B01] Forward+: Bringing Deferred Lighting to the Next Level
Takahiro Harada; Jay McKee; Jason C. Yang. 2012. Eurographics 2012 Short Papers.
Use: Construct per-tile light lists, then use them in forward material shading. **Limit:** A depth prepass, list construction and synchronization add costs.
Open primary source | doi.org | High when local-light overlap justifies culling

[B02] Clustered Deferred and Forward Shading
Ola Olsson; Markus Billeter; Ulf Assarsson. 2012. High Performance Graphics.
Use: Group samples by three-dimensional position, optionally normal, to avoid irrelevant light evaluations; depth clustering helps at discontinuities. **Limit:** Cluster storage, construction, occupancy and overflow require management.
Open primary source | research.chalmers.se | High after a simple forward baseline

[B03] Stochastic Tile-Based Lighting in HypeHype
Jarkko Lempiäinen. 2025. SIGGRAPH Advances in Real-Time Rendering in Games course.
Use: Investigate bounded sampled-light shading for difficult creator scenes. **Limit:** G-buffer, shadows, noise and temporal work remain.
Open primary source | advances.realtimerendering.com | Research branch for unpredictable many-light scenes

[B04] Understanding Render Passes
Arm. 2021. Arm technical guide, document 102479_0100_02_en.
Use: Understand attachment loads, stores and render-pass boundaries so intermediate data need not make avoidable trips to external memory. **Limit:** Native API examples and device-specific behavior must be translated to capabilities actually exposed by browser APIs.
Open primary source | documentation-service.arm.com | Essential architecture reading

[B05] Tile-Based Rendering
Arm. Living documentation. Arm technical guide.
Use: Explains the role of tile memory and why retaining intermediate per-pixel data on chip can reduce bandwidth. **Limit:** Hardware tile-based rendering is different from an engine choosing deferred shading.
Open primary source | documentation-service.arm.com | Essential mobile GPU background

[B06] WebGPU
W3C GPU for the Web Working Group. 2026. W3C Candidate Recommendation Draft, 15 September 2026.
Use: Source of truth for capabilities, formats, sample counts and limits. **Limit:** Core and Compatibility Mode differ; optional features require negotiation.
Open primary source | www.w3.org | Essential implementation reference

[B07] WebGL & WebGPU Updates
Ken Russell; Kai Ninomiya. 2026. GDC 2026, on behalf of the WebGL and WebGPU Working Groups.
Use: Documents browser-family availability, WebGPU Compatibility Mode, the stable C API and the distinction between shipped and future capabilities. **Limit:** Availability is a dated snapshot across differing operating-system and GPU subsets; do not turn browser-family support into universal device support.
Open primary source | www.khronos.org | Essential current platform context

[B08] WebGPU Explainer
GPU for the Web Community Group. 2026. Draft Community Group Report, current opened revision dated 6 November 2026.
Use: Explains WebGPU's design goals, explicit resource/pipeline model and relationship to modern native graphics APIs. **Limit:** An explainer is not the normative capability contract; use the specification and runtime feature detection for implementation decisions.
Open primary source | gpuweb.github.io | Architecture reference

[B09] WebGL best practices
MDN contributors. Living documentation. MDN living documentation.
Use: Practical guidance on memory, resolution, batching, blocking calls, framebuffers, texture compression and context loss. **Limit:** WebGL-specific extensions and procedures do not automatically apply to WebGPU; performance still needs measurement on target devices.
Open primary source | developer.mozilla.org | Essential fallback and browser performance guidance

[B10] KTX File Format Specification, Version 2
Khronos Group. 2020. Khronos specification, initially ratified 14 August 2020; living revision.
Use: Supports supercompressed texture distribution and small-mip-first streaming, helping separate transfer format from GPU-resident format. **Limit:** Compression formats, HDR encodings and transcoding targets have different capabilities and error characteristics; validate the chosen lighting payload.
Open primary source | github.khronos.org | Essential asset-delivery architecture

[B11] KHR_texture_basisu
Khronos Group. 2021. glTF 2.0 ratified extension; living specification.
Use: Defines KTX 2.0 textures with Basis Universal supercompression in glTF, allowing efficient transmission and transcoding to supported GPU formats. **Limit:** Do not assume one encoding is suitable for all color, normal, scalar and HDR lighting data; inspect artifacts and format support.
Open primary source | github.com | High for portable compressed assets

[B12] A Survey of Temporal Antialiasing Techniques
Lei Yang; Shiqiu Liu; Marco Salvi. 2020. Computer Graphics Forum 39(2), pages 607-621.
Use: A foundation for sample accumulation, history validation, temporal upsampling and the quality challenges shared by temporal rendering effects. **Limit:** TAA is not a free general-purpose denoiser.
Open primary source | research.nvidia.com | High before introducing temporal lighting dependencies

[B13] SMAA: Enhanced Subpixel Morphological Antialiasing
Jorge Jimenez; Jose I. Echevarria; Tiago Sousa; Diego Gutierrez. 2012. Eurographics / Computer Graphics Forum 31(2).
Use: Improves morphological edge reconstruction and offers combinations with multisampling and temporal reprojection. **Limit:** A spatial-only variant does not reconstruct missing temporal samples or remove stochastic lighting noise; include its extra passes in bandwidth measurements.
Open primary source | www.iryoku.com | High as a non-temporal image-quality baseline

[B14] WebGPU Samples
WebGPU community contributors. Living documentation. Living reference implementation collection.
Use: Runnable examples for shadow mapping, MSAA, render bundles, timestamps, workers, reversed-Z and deferred rendering. **Limit:** Teaching samples are not demonstrated optimal engine architectures; benchmark and adapt resource lifetime, batching and error handling.
Open primary source | webgpu.github.io | High implementation reference

[B15] Optimizing WebGL
Emscripten contributors. Living documentation. Emscripten living documentation.
Use: Explains API overhead, batching, emulation costs and avoiding CPU/GPU stalls when porting native rendering workloads. **Limit:** Primarily WebGL guidance.
Open primary source | emscripten.org | High for compiled-language browser integration

Occlusion, atmosphere, temporal rendering and further research

[R01] Scalable Ambient Obscurance
Morgan McGuire; Michael Mara; David Luebke. 2012. High Performance Graphics.
Use: Architecture-aware ambient obscurance, depth prefiltering and reconstruction reduce bandwidth and make sample work predictable. **Limit:** Screen-space visibility is incomplete; count the depth/normal preparation and reconstruction passes.
Open primary source | research.nvidia.com | Useful AO baseline

[R02] Practical Real-Time Strategies for Accurate Indirect Occlusion
Jorge Jimenez; Xian-Chun Wu; Angelo Pesce; Adrian Jarabo. 2016; revised technical memo 2019/2020. SIGGRAPH 2016 presentation; Activision technical memo ATVI-TR-19-01.
Use: GTAO provides a principled ambient-occlusion approximation; the work also addresses multibounce compensation and specular occlusion. **Limit:** AO is not a general GI solver.
Open primary source | research.activision.com | Optional AO quality upgrade

[R03] XeGTAO
Filip Strugar; Steve McCalla / Intel. 2021. Author implementation, living repository.
Use: Practical GTAO implementation with depth processing, main evaluation and denoising stages; useful reference for integration decisions. **Limit:** HLSL/DirectX implementation requires adaptation and actual WebGPU measurement.
Open primary source | github.com | Implementation study

[R04] Efficient GPU Screen-Space Ray Tracing
Morgan McGuire; Michael Mara. 2014. Journal of Computer Graphics Techniques 3(4), 73-85.
Use: Perspective-correct DDA walks a depth buffer without redundant oversampling; includes implementation details and GLSL. **Limit:** Hidden and off-screen surfaces remain unavailable.
Open primary source | www.jcgt.org | Optional reflections/contact effects

[R05] A Scalable and Production Ready Sky and Atmosphere Rendering Technique
Sebastien Hillaire. 2020. Eurographics Symposium on Rendering / Computer Graphics Forum.
Use: A small collection of lookup tables supports dynamic atmosphere and aerial perspective with scalable evaluation. **Limit:** A physically based atmosphere is more engineering than a gradient sky; clouds are a separate workload.
Open primary source | sebh.github.io | High value after simple fog

[R06] Physically-Based and Unified Volumetric Rendering in Frostbite
Sebastien Hillaire. 2015. SIGGRAPH Advances in Real-Time Rendering course.
Use: Unifies participating media, light scattering and shadowing using a volumetric representation. **Limit:** Volume resolution, lighting, integration and history add memory and runtime work; cheap height fog should come first.
Open primary source | www.ea.com | Optional high tier

[R07] High Quality Temporal Supersampling
Brian Karis. 2014. SIGGRAPH Advances in Real-Time Rendering course.
Use: A production foundation for temporal sampling, reprojection, accumulation and history handling. **Limit:** Good motion vectors and history rejection are essential; softness and ghosting matter for stylized art.
Open primary source | de45xmedrsdbp.cloudfront.net | Study before temporal effects

[R08] Mipmapping Normal Maps
Michael Toksvig. 2005. Journal of Graphics Tools 10(3), 65-71.
Use: Uses normal variation to reduce sparkling and aliasing of specular highlights as normal detail becomes subpixel. **Limit:** Adapt the idea consistently to the chosen BRDF and texture encoding; old shininess formulas are not interchangeable with roughness.
Open primary source | www.tandfonline.com | High value material filtering

[R09] LEAN Mapping
Marc Olano; Dan Baker. 2010. ACM I3D.
Use: Stores filterable normal-distribution information for stable highlights and layered normal detail. **Limit:** Extra texture information and shader complexity may outweigh benefit for deliberately broad, simple highlights.
Open primary source | userpages.cs.umbc.edu | Advanced normal/specular filtering

[R10] LightOpt: Lights Optimization for Real-time Rendering
Tuo Chen; Luyan Cao; Kui Wu; Shimin Hu. 2026. SIGGRAPH Conference Papers, article 48.
Use: Differentiable offline optimization removes, merges and inserts lights while preserving appearance and supporting time-of-day setups. **Limit:** Optimizes a chosen appearance objective, not arbitrary interactive semantics; published lighting-pass speedups are scene-specific.
Open primary source | doi.org | High value toolchain research

[R11] Spatiotemporal Reservoir Resampling for Real-time Ray Tracing with Dynamic Direct Lighting
Benedikt Bitterli; Chris Wyman; Matt Pharr; Peter Shirley; Aaron Lefohn; Wojciech Jarosz. 2020. ACM TOG / SIGGRAPH.
Use: ReSTIR reuses candidate light samples over space and time to make many-light Monte Carlo estimation practical. **Limit:** Visibility evaluation, reservoirs, temporal validation and denoising remain costs; it is not a portable ray-tracing API.
Open primary source | research.nvidia.com | Advanced many-light foundation

[R12] Lumen: Real-time Global Illumination in Unreal Engine 5
Daniel Wright et al.. 2022. SIGGRAPH Advances in Real-Time Rendering course.
Use: Study the decomposition into scene representations, tracing, radiance caches, reconstruction and temporal reuse. **Limit:** A complete dynamic-GI system is a large dependency chain.
Open primary source | advances.realtimerendering.com | Architecture study

[R13] Global Illumination Based on Surfels
Henrik Halen; Andreas Brinck. 2021. SIGGRAPH Advances in Real-Time Rendering course.
Use: Surface elements cache and amortize dynamic indirect lighting; useful when investigating surface-based illumination caches. **Limit:** Placement, acceleration, tracing, updates and reconstruction are substantial systems, not just a surfel texture.
Open primary source | www.ea.com | Research track

[R14] Stupid Spherical Harmonics (SH) Tricks
Peter-Pike Sloan. 2008; updated 2010. GDC technical notes.
Use: Practical SH evaluation, products, ringing and representation conventions complement the original irradiance paper. **Limit:** Coefficient ordering, normalization and convolution conventions must match throughout the engine.
Open primary source | www.ppsloan.org | Essential implementation reading

[R15] Efficient Irradiance Normal Mapping
Ralf Habel; Michael Wimmer. 2010. ACM I3D, 189-195.
Use: A hemispherical basis uses six coefficients to represent directional irradiance over the surface-relevant hemisphere. **Limit:** Requires consistent hemisphere frames and a compatible baker; compare against simpler reference encodings.
Open primary source | www.cg.tuwien.ac.at | Directional encoding alternative

[R16] Adaptive Probe Volumes: concepts
Unity Technologies. Living documentation. Official engine documentation.
Use: Adaptive probe density, streaming and lighting scenarios illustrate practical large-scene lighting-cache organization. **Limit:** Engine-specific implementation and constraints are not a ready-made browser design.
Open primary source | docs.unity.com | Production architecture study

[R17] Real-time Global Illumination by Precomputed Local Reconstruction from Sparse Radiance Probes
Ari Silvennoinen; Jaakko Lehtinen. 2017. ACM TOG / SIGGRAPH Asia.
Use: Precomputed local visibility-aware reconstruction combines sparse probes with dynamic lighting in mostly static scenes. **Limit:** Geometry assumptions and precomputation remain; benchmark the complete update and reconstruction path.
Open primary source | arisilvennoinen.github.io | Advanced relighting option

[R18] A Distributed, Decoupled System for Losslessly Streaming Dynamic Light Probes to Thin Clients
Michael Stengel; Zander Majercik; Benjamin Boudaoud; Morgan McGuire. 2021. arXiv 2103.05875.
Use: Computes indirect lighting remotely while primary visibility and direct lighting stay local, decoupling display latency from probe updates. **Limit:** Network cost, server capacity, update latency and browser codec support need independent design; not a drop-in WebGPU implementation.
Open primary source | arxiv.org | Strategic server-assisted option

[R19] Physically Based Rendering in Filament
Romain Guy; Mathias Agopian / Google. Living documentation. Official renderer documentation.
Use: Integrated reference for physically motivated materials and lighting conventions. **Limit:** Choose a small feature set for the target art style.
Open primary source | google.github.io | Essential implementation reference

[R20] Towards Real-time Dynamic GI on Mobile
Jasper de Winther. 2026. SIGGRAPH Moving Mobile Graphics course contribution.
Use: Recent mobile-GI work connects renderer research to mobile architecture constraints. **Limit:** The primary contribution page was verified; this review does not establish a portable browser implementation or benchmark.
Open primary source | traverse.nl | Research watchlist

[R21] SLIM: Scaling User-Generated 3D Worlds on Roblox
Sergey Makeev. 2026. SIGGRAPH Advances in Real-Time Rendering course.
Use: Device-adaptive runtime representations are relevant when user-authored worlds exceed small-device budgets. **Limit:** An asset/runtime representation system, not a lighting algorithm; consult full materials before copying production assumptions.
Open primary source | www.advances.realtimerendering.com | Creator-world architecture study

[R22] Screen Space Indirect Lighting with Visibility Bitmask
Olivier Therrien; Yannick Levesque; Guillaume Gilet. 2022; arXiv version 2023. The Visual Computer; arXiv 2301.11376.
Use: A visibility bitmask improves thin-surface behavior for horizon-based ambient and indirect diffuse lighting. **Limit:** Unknown off-screen and hidden surfaces remain missing; reconstruction, temporal work and double counting need explicit treatment.
Open primary source | arxiv.org | Optional screen-space bounce